1. Lower Bound
2. Upper Bound
3. Search Insert position
4. Floor and Ceil in Sorted Array
5. First and last occurrence of x
6. Count occurrences of a number in a sorted array with duplicates.
7. Search in Sorted Rotated Array - I (unique els)
8. Search in Sorted Rotated Array - II (Duplicate els)
9. Find min in Sorted Rotated Array
10. Find out how many times has an array been rotated
11. Single Element in Sorted Array
12. Find peak Element
13. Floor in Sorted Array
14. Find square root of a number in log N
15. Find the Nth root of an integer
16. Koko Eating Bananas
17. Minimum Number of Days to make m bouquets
18. Find the smallest Divisor
19. Capacity to Ship Packages within D Days.
20. Kth missing positive number
21. Aggressive Cows
22. Allocate Books

Painters partition

23. Split Array - Largest Sum
24. Minimize max distance to gas station.
25. Median of two Sorted Arrays
26. Kth element of two Sorted Arrays
27. Find the row with max number of 1's.
28. Search in a 2D matrix.
29. Search in a 2D matrix - II.
30. Find peak element in a 2D matrix
31. Matrix median
32. Minimize the maximum difference of Pairs
33. Find minimum in Sorted Rotated Array - II
34. Fair Candy Swap

— Binary Search —

=> Binary Search - (Sorted) Binary Search Algorithm is a searching algorithm used in a sorted array by repeatedly dividing the search interval in half. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to O(logN).

Iterative :-

```
import java.util.*;
public class TUF {
    public static int binarySearch(int nums[], int target){
        int n = nums.length;
        int low = 0, high = n-1;

        while(low <= high){
            int mid = (low+high)/2;
            if(nums[mid] == target) return mid;
            else if(target > nums[mid]){
                low = mid+1;
            }
            else{
                high = mid-1;
            }
        }
        return -1;
    }
}
```

TC- O(logN)

Recursive:-

```
public static int binarySearch(int nums[], int low,
    int high, int target){

    if (low > high) {    // Base case
        return -1;
    }

    int mid = (low + high)/2;
    if (nums[mid] == target){
        return mid;
    }
    else if (target > nums[mid]) {
        return binarySearch(nums, mid+1, high,
            target);
    }
    return binarySearch(nums, low, mid-1, target);
}

public static int search(int nums[], int target){
    return binarySearch(nums, 0, nums.length-1,
        target);
}
```

TC - O(logN)

PAGE NO
DATE

DRY RUN:—

arr[] = [2, 3, 4, 5, 6] target => 4

(0 1 2 3 4)

low, high

(1) low = 0 , high = n-1

mid = 0 + (4-0)/2 => 0 + 4/2 => 2

mid

[2, 3, 4, 5, 6]

0 1 2 3 4

mid == target return

(2) then Mid < target {if target = 5}

low = mid+1

[2, 3, 4, 5, 6]

low, high

mid == target

5 == 5 return 5 as an answer

{ O(logN) => Time Complexity }

(01) Lower Bound :-

arr[] = [3, 5, 8, 15, 19, 19, 19] n = 7 , x = 19
        0 1 2 3 4 5 6

arr[idx] >= x (first index return)

Ans: 4

DRY RUN:- [0 1 2 3 4 5 6]
[3, 5, 8, 15, 19, 19, 19] x = 19
low mid high

⇒ mid = low + (high - low) / 2
= 0 + (6 - 0) / 2 ⇒ 3

⇒ mid < 19 ⇒ low + 1

⇒ [3, 5, 8, 15, 19, 19, 19]
low high

⇒ mid = 4 + (6 - 4) / 2

⇒ 4 + 1 ⇒ 4

mid >= x ⇒ ans ⇒ mid

⇒ 4 index

Code:- Brute force:-

```
for (int i = 0; i<n; i++) {
    if (arr[i] >= x) {
        return i;
    }
}
return n;
```

TC - O(N)
SC - O(1)

Code:- Optimal (Binary Search)-

```
int low = 0, high = n-1;
int ans = n; (indicates index is out of bound
              -there is no value present in array)
while (low <= high) {
    int mid = (low + high)/2;

    if (arr[mid] >= x) {
        ans = mid;
        high = mid-1;
    }
    else {
        low = mid+1;
    }
    return ans;
}
```

TC - $O(\log_2 n)$
SC - O(1)

(02) Upper Bound :- arr[idx] > x

Smallest index such that

arr[] = [2, 3, 6, 7, 8, 8, 11, 11, 11, 12] n=10, x=8

DRY RUN :- [2, 3, 6, 7, 8, 8, 11, 11, 11, 12] n=10

(1) low, mid, high

0+9 | 2 ⇒ 4 ⇒ mid

~~mid~~ if (mid > x) ans = mid

~~high = low - 1~~

low = ~~high~~ + 1

8 > 8 (X)

(2) [~~8~~, ~~8~~, ~~11~~, ~~12~~, ~~12~~]

~~low~~, ~~mid~~ | high, low, mid, ~~high~~

low + high

4 + 4 ⇒ 8/2 ⇒ 4

~~mid < x~~ mid > x ans = 2 (index)

~~low = mid + 1~~ high = mid - 1

(3) [~~8~~, ~~11~~ ~~11~~, ~~12~~]

2 3

2+3/2 ⇒ 5/2 ⇒ 2

high, low, high, mid

mid

PAGE NO.
DATE: / /

① [2, 3, 6, 7, 8, 8, 11, 11, 11, 12]
(0 1 2 3 4 5 6 7 8 9)
low, mid, high

mid < x
low = mid + 1

② [8, 8, 11, 11, 11, 12]
(4 5 6 7 8 9)
low, mid, high

5 + 9 / 2 ⇒ 14/2 ⇒ 7

mid > x
ans = mid = 7
high = mid - 1

③ [8, 11, 11]
(5 6 7)
low, high, mid

5 + 6 ⇒ 11/2 ⇒ 5

mid < x
low = mid + 1

④ [11, 11]
(6 7)
low, high, mid

⇒ 6 + 6 ⇒ 12/2 ⇒ 6

mid > x
ans ⇒ 6

Crossed

Code :-

```
int low = 0, high = n-1;
int ans = n;

while (low <= high) {
    int mid = (low+high)/2;

    if (arr[mid] > x) {
        ans = mid;
        high = mid-1;
    }
    else {
        low = mid+1;
    }
}

return ans;
}
```

TC - $O(\log_2 n)$

SC - O(1)

⇒ When the loop ends (low > high), we haven't found the exact target but -

- All elements before low are less than target.
- All elements at or after low are greater than or equal to target.

(03) Search Insert position -

Ex-(01) Input :- nums = [1, 3, 5, 6], target = 5

output :- 2

Ex(02) Input :- nums = [1, 3, 5, 6], target = 7

output :- 4

```
int low = 0, high = nums.length - 1;

while ( low <= high) {
    int mid = low + (high - low) / 2;

    if (mid == target) return mid;

    else if (nums[mid] > target) {
            high = mid - 1;
    }
    else {
            low = high + 1; mid + 1;
    }
}
return low;
```

- low is always the lowest possible index where the target could be.

TC -	O(log N)
SC -	O(1)

- high is the highest possible index where the target could be.

(04) Floor and Ceil in Sorted Array -

Ex: Input :- n = 6 arr[] = { 3, 4, 4, 7, 8, 10 }, x = 5

Result :- 4, 7

Explanation :- The floor of 5 in the array is 4, and the ceiling of 5 in the array is 7.

```
Static int findFloor(int arr[], int n, int x){
  int low = 0, high = n-1, ans = -1;

  while (low <= high){
    int mid = (low+high)/2;

    if(arr[mid] <= x){
      ans = arr[mid];
      low = mid+1;
    }else{
      high = mid-1;
    }
  }

  return ans;
}

Static int findCeil(int arr[], int n, int x){
  int low = 0, high = n-1, ans = -1;

  while (low <= high){
    int mid = (low+high)/2;

    if(arr[mid] >= x){
      ans = arr[mid];
      high = mid-1;
    }
    else{
      low = mid+1;
    }
  }
  return ans;
```

TC - 2 O(log n)
SC - O(1)

(05) First and last occurrence of x in sorted Array. (return index)

ans = [2, 4, 6, 8, 8, 8, 11, 13] (indices 0 1 2 3 4 5 6 7)

n = 8 (target)

output = {3, 5}

Brute force :- Linear Search

Input :- [2, 4, 6, 8, 8, 8, 11, 13] (indices 0 1 2 3 4 5 6 7)

→ linearly search ——— n = 8 (target)

① [2, 4, 6, 8, 8, 8, 11, 13]
first ↑

first ≠ 8

② [2, 4, 6, 8, 8, 8, 11, 13]
↓
first

④ [2, 4, 6, 8, 8, 8, 11, 13]
↓
first

⑤ [2, 4, 6, 8, 8, 8, 11, 13] (indices 0 1 2 3 4 5 6 7)
↓
first

first == -1 ⇒ first == 3
last = 3

Code:-

```
first = -1 , last = -1

for ( i = 0 → n-1) {
   if (arr[i] == x) {
        if (first == -1)  first = i;
        last = i
   }
}
```

TC - O(N)
SC - O(1)

(6) [2, 4, 6, 8, 8, 8, 11, 13]

↓ i

first ≠ -1 arr[i] == target

last = 4

(7) [2, 4, 6, 8, 8, 8, 11, 13]

↓ arr[i]

first ≠ -1 , arr[i] == target

last = 5

return {3, 5}

optimal approach :- (upper bound + lower bound)

	0	1	2	3	4	5	6	7	
arr =	2,	4,	6,	8,	8,	8,	11,	13	x = 8
			lb				up		

lower bound - arr[idx] >= x

Upper bound - arr[idx] > x

x = 14 ⇒ (lb(14) == x) || (arr[lb] != x)

low = 0 high = 7

① mid = 0 + (7-1)/2 = 3

mid(3) = 8

② (8 == 8) ~~return~~ ans = 3

high = mid - 1

③ low = 0 high = 2

mid = 0 + (2-1)/2 = 0

mid(0) = 2

④ 2 < 8 low = mid + 1

low = 1 high = 2

⑤ Mid = 1 + (2-1)/2 = 1

Mid(1) = 4 < 8

low = mid + 1

⑥ low = 2 high = 2 ans = 3rd index

mid = 2 + (2-2)/2 ⇒ 2

nums[2] = 6 (low > high)

6 < 8 low = mid + 1 == 3 (stop)

PAGE NO.
DATE: / /

Upper bound :-

① low = 0 high = 7 mid = 0 + (7-0)/2 = 3
Mid[3] = 8
8 == 8 (x) (x > target)
low = mid + 1

② low = 4 high = 7 mid = 4 + (7-4)/2 = 5
Mid[5] = 8
8 == 8 (x) (x > target)
low = mid + 1

③ low = 6 high = 7 mid = 6 + (7-6)/2 = 6
Mid[6] = 11
(11 > target) ans = mid == 6

return ans == 6

main func

① if (n == 0) return new int[] {-1, -1};

② int lower = lb(nums, target);
int upper = ub(nums, target) - 1;

handling for n out of bound err

③ if (lower >= n || nums[lower] != target) {
return new int[] {-1, -1};
else

④ return new int[] { lower, upper };

Code:-

```
public int lb(int nums[], int target){
    ---
    --- lower bound ---          arr[mid] >= x
    ---- Code will be here ----
}

public int ub(int nums[], int target){
    ---------
    --- upper bound ---- (arr[mid] > x).
    ---- Code will be here ---
}

public int[] searchRange(int nums[], int target){
    int n = nums.length;
    if(n==0){
        return new int[]{-1,-1};
    }

    int lowerBound = lb(nums, target);
    int upperBound = ub(nums, target) - 1;

    if(lower >= n || nums[lower] != target){
        return new int[]{-1,-1};
    }

    return new int[]{lower, upper};
}
```

TC - $2 * O(\log_2 N)$
SC - $O(1)$

Plain BS :-

```
public int[] searchRange(int[] nums, int target) {
    int first = firstOccurance(arr, n, k);
    if (first == -1) return new int[]{-1, -1};

    int last = lastOccurance(arr, n, k);
    return {first, last};
}
```

fn (int arr[], n, x) { { first }
low = 0, high = n-1;
first = -1

while (low <= high) {
mid = low + (high - low) / 2;

if (arr[mid] == x) {
first = mid, high = mid-1;
} else if (arr[mid] < x) {
low = mid+1;
} else {
high = mid-1;
}
~~return first~~
}

fn(int arr[], n, x) {
low = 0, high = n-1;
last = -1;

while (low <= high) {
mid = low + (high-low) / 2;
if (arr[mid] == x) {
last = mid,
low = mid+1;
} else if (arr[mid] < x) {
low = mid+1;
} else {
high = mid-1;
}
}
~~return last~~,

2.

(06) Count occurrences of a number in a sorted array with duplicates.

Ex - 01. N=7, X=3, arr[] = [2, 2, 3, 3, 3, 3, 4]
output ⇒ 4

Brute force - cnt ⇒ 0

```
for (int i = 0; i < n; i++) {
    if (arr[i] == x) {
        cnt = cnt + 1;
    }
}
return cnt;
```

① [2, 2, 3, 3, 3, 3, 4] X=3
 i

② [2, 2, 3, 3, 3, 3, 4] 2 ≠ 3
  i

③ [2, 2, 3, 3, 3, 3, 4] 3 = 3
    i cnt = 1

④ [2, 2, 3, 3, 3, 3, 4] 3 = 3
     i cnt = 2

⑤ [2, 2, 3, 3, 3, 3, 4, 4] 3 = 3
      i cnt = 3

TC - O(N)
SC - O(1)

Optimal Approach using Binary Search –

Ex– [2, 2, 3, 3, 3, 3, 4] (0 1 2 3 4 5 6) x = 3.

(1) find lowerBound

(2) find upperBound

(3) (lowerBound idx + upperBound idx) – 1

= number of count of the x.

(1) [2, 2, 3, 3, 3, 3, 4] (0 1 2)

↓

It's lower Bound will give index number 2.

(2) [2, 2, 3, 3, 3, 3, 4] (0 1 2 3 4 5 6)

It's upperBound will give the index number 6

Now 6 – 2 ⇒ 4

The number of appearance of 3 is 4.

Code:-

```
public int lower(int arr[], int x){
    int n= arr.length;
    int low = 0
    int high = n-1; int ans = n;

    while( low <= high){
        int mid = low + (high-low)/2;

        if (arr[mid] >= x){
            ans = mid
            high = mid-1;
        }
        else {
            low = mid+1;
        }
    }
    return ans;
}

public int upper(int arr[], int x){
    int n= arr.length;
    int low= 0;
    int high= n-1;
    int ans= n;

    while ( low <= high ){
        int mid = low + (high-low)/2;
```

~~while (true~~

```
if (arr[mid] > x) {
    ans = mid;
    high = mid-1;
}
else {
    low = mid+1;
}
}
return ans;
}
```

~~TC~~ -

~~SC~~ -

```
public int[] countOccurence(int arr[], int x) {
    int n = arr.length;

    int lowerBound = lower(arr, x);
    ~~if (lowerBound == n) return new int[] {-1, -1};~~

    int upperBound = upper(arr, x);
    ~~int ans = (lowerBound + upperBound) - 1;~~
    ~~return new int[] {ans};~~
    if (lowerBound == n || arr[lowerBound] != target) {
        return 0;
    }

    return upperBound - lowerBound;
}
```

TC = 2 * O(logN)

SC = O(1)

(07) Search in Rotated Sorted Array - I (Unique Element)

Ex-① Input :- nums = [4,5,6,7,0,1,2], target = 0
output :- 4 (index return)

Brute force :- ① We will search for the element by linear search in an array.

Code :-

```
public static int search(arr[], int k){
    int n = arr.length;

    for(int i=0; i<n; i++){
        if(arr[i] == k){
            return i;
        }
    }
    return -1;
}
```

Time Complexity - O(N)

Space Complexity - O(1)

(Optimal):- Binary Search

[4, 5, 6, 7, 0, 1, 2] k = 0 ans = 4

0 1 2 3 4 5 6

① [4, 5, 6, 7, 0, 1, 2] mid = $\frac{0+6}{2} \Rightarrow 3$

left mid right

Mid >= left (sorted)

~~mid~~

left > ~~mid~~ target ⇒ left = mid+1;

② [7, 0, 1, 2] mid = $\frac{4+6}{2} \Rightarrow 5$

left mid right

Mid ≠ target (k)

nums[left] <= nums[mid] (sorted)

nums[left] <= target && target <= arr[mid]

right = mid-1;

③ [7, 0] mid = $\frac{4+4}{2} \Rightarrow$ ④

left

right

mid

PAGE NO
DATE : / /

mid == target (k)

arr(mid) == 0 1

return index (mid);

Ex- ① nums = [4, 5, 6, 7, 0, 1, 2]

Input) target = 3

output = -1

Ex: ② nums = [1], target = 0

output = -1;

Return -1 as an output if there is no element exists.

Code:-

```
public int search (int nums[], int target) {
    int n= nums.length;
    int left = 0;
    int right = n-1;

    while ( left <= right ) {
        int mid = left + (right - left)/2;

        if (nums[mid] == target ) {
            return mid;
        } // if left is sorted
        else if (nums[left] <= nums[mid] ) {
            if (nums [left] <= target &&
                target <= nums[mid ] ) {
                right= mid-1;
            } else{
                left = mid+1;
            }
        } else {
            if (nums[mid] <=target &&
                nums [right] >= target ) {
                left = mid+1;
            } else{
                right = mid-1;
            }
        }
    }
    return -1;
}
```

TC -	O(logN)
SC -	O(1)

PAGE NO.
DATE: / /

(08) Search in Rotated Sorted Array - II (Duplicates Allowed)

return "yes" or "no"

arr[] = [7, 8, 1, 2, 3, 3, 3, 4, 5, 6] target = 3

Ex - [3, 1, 2, 3, 3, 3, 3]

{arr[low] == arr[mid] == arr[high]

[Try to Shrink it]

Brute force :- worst = O(n/2)

```
int n = arr.length;

for (int i = 0; i < n; i++) {
    if (arr[i] == k) {
        return i;
    }
    return -1;
}
```

TC - O(N)
SC - O(1)

Edge Case:- By comp cannot part is
arr[] = [3, 1, 2, 3, 3,
low mid

Optimal Approach (Binary Search) :-

Edge case:- arr = [3, 1, 2, 3, 3, 3, 3]
low mid high

(Try to trim down this condition)
arr[low] == arr[mid] == arr[high]

Trim
arr[] = [3, 1, 2, 3, 3, 3, 3]
low mid high

(Continue करने पर वही से code Stop होकर वापस से Starting से चलने लगता है)

TC - $O(\log_2 N)$ → average
TC ≈ $O(N/2)$
arr[] = [3, 3, 1, 3, 3, 3, 3]

Ex:- (1) [3, 3, 1, 3, 3, 3, 3] target = 1
0 1 2 3 4 5 6
low mid high

nums[mid] != target

① nums[mid] ~~==~~ nums[low]

② ~~nums~~ target < nums[low]
nums[mid] > target

it means target lies in the left half.

high = mid - 1

(2) [3, 3, 1, 3]
0 1 2 3
low mid high

(1) nums[mid] != target

(2) nums[high] >= nums[mid]

(3) nums[high] >= target

(4) nums[mid] <= target

it means target lies in the right half

low = mid + 1

(3) [3, 3, 1]
high
low
mid

nums[mid] == target

Return "true".

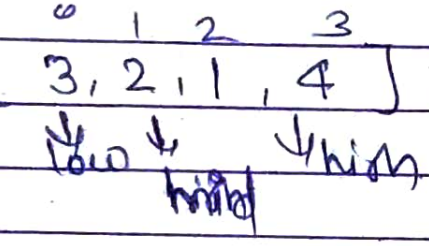
PAGE NO
DATE

① If there is same problem where one is having unique element and other is having duplicate element.

② You have to see where the duplicate elements problem is failing and try to resolve the problem by using unique elements problem code.

In Binary Search Problems

Ex.

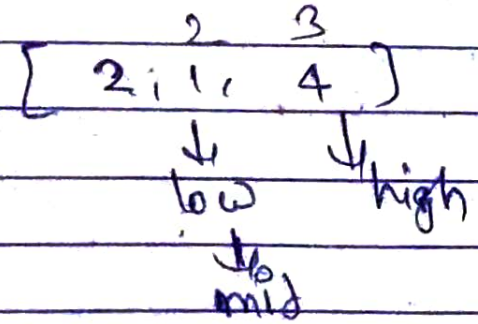


target = 5

(mid) != target it will lier on right part.

(mid < high) ⇒

low = mid+1



5/2 ⇒ (2)

(mid != target)...

PAGE NO.
DATE: / /

Code:-

```
int low = 0;
int high = n-1;

while (low <= high) {
    int mid = low + (high-low)/2;

    if (nums[mid] == target) {
        return true;
    }
    else if (nums[mid] == nums[low] && nums[mid]
                == nums[high]) {
        low = low+1;
        high = high-1;
        continue;
    }
    else if (nums[mid] >= nums[low]) {
        if (nums[mid] >= target &&
            nums[low] <= target) {
            high = mid-1;
        }
        else {
            low = mid+1;
        }
    } else if (nums[mid] <= nums[high]) {
        if (target >= nums[mid] &&
            target <= nums[high]) {
            low = mid+1;
        }
        else {
            high = mid-1;
        }
    }
    return false;
```

(09) Find minimum in Rotated Sorted Array - [unique]

arr[] = [4, 5, 6, 7, 0, 1, 2] min = 0 Ans →

Brute force:-

```
min = Integer.MAX_VALUE;
for (int i=0; i<n; i++) {
    if (arr[i] < min) {
        min = arr[i];
    }
}
return min;
```

TC - O(N)
SC - O(1)

(Optimal) -- Using Binary Search -

Input :- [7, 8, 1, 2, 3, 4, 5, 6]

ans ⇒ 1

arr[] = [7, 8, 1, 2, 3, 4, 5, 6]

① ans = Integer.MAX_VALUE;

Now there is 2(mid) ---- 6 (high) part is sorted. (right part is sorted min == mid)

Take the min from them which is mid == 2 & Eliminate it.

high = mid-1;

② arr[] = [7, 8, 1]

Now, Here is 7--(low) --- 8(mid) part is sorted.

Left part is sorted here, min == left (7)]

ans = 2 < 7 (Do not update)

left = mid+1

③ [1]

high
left
mid

ans = 1 (Updated) ans.

Code:-

```
int low = 0, high = arr.length - 1;
int ans = Integer.MAX_VALUE;

while (low <= high) {
    int mid = low + (high-low) / 2;

    // if the arr is sorted
    // if this entire part is sorted
    if (arr[low] <= arr[high]) {
        ans = Math.min (ans, arr[low]);
        break;
    }

    // if the left part is sorted
    if ( arr[low] <= arr[mid]) {
        ans= Math.min ( ans, arr[low]);
        low = mid+1;  // eliminate the left half
    }
    else {
        ans = Math.min (ans, arr[mid]);
        high = mid-1;
    }
}
return ans;
}
```

TC - O(logN)
SC - O(1)

(10) Find out how many times has an array been rotated -

Ex: arr[] = [3, 4, 5, 1, 2] ans = 3

[Eventually the number of rotation will be the index of smallest element in an array.]

(1) arr[] = [3, 4, 5, 1, 2] index = -1
0 1 2 3 4
low mid high

3 --- 5 is sorted min = low (3)
Eliminate entire part
low ~~left~~ = mid + 1

(2) arr[] = [3, 4, 5, 1, 2] 3+4/2 ⇒ 3
low high
mid

index

1 --- 2 is sorted

index = low

Brute force -
TC - O(N)
SC - O(1)

Return index as an answer.
Brute.

Brute:- int n = arr.length; int ans = Integer.MAX_VALUE;
index = -1
for (int i = 0; i < n; i++) {
if (arr[i] < ans) {
ans = arr[i]
index = i;
} }

return index;

Code:-

```
public static int findKRotation(int arr[]){
    int low = 0, high = arr.length-1;
    int ans = Math Integer.MAX_VALUE;
    int idx = -1

    while(low <= high){
        int mid = low + (high-low)/2;

        if(arr[low] <= arr[high]){
            if(arr[low] < ans){
                idx = low;
                ans = arr[low];
            }
            break;
        }
        // if left past is sorted
        if(arr[low] <= arr[mid]){
            if(arr[low] < ans){
                idx = low;
                ans = arr[low];
            }
            low = mid+1;
        } else{
            if(arr[mid] < ans){
                idx = mid;
                ans = arr[mid];
            }
            high = mid-1;
    } }
    return idx;
}
```

TC- O(logN)
SC- O(1)

(ii) Single Element in Sorted Array – (return index

arr[] = [1,1,2,2,3,3,4,5,5,6,6] output :-

Brute force :- DRY RUN –

① 0^1 = Same no. (if xor with 0).

② 1^1 = 0

③ 0^2 = 2
2^2 = 0
0^3 = 3
3^3 = 0
0^4 = 4
4^5 = 1
1^5 = 4
4^6 = 2
2^6 = (4) ← Ans:

which is the single element in the given array.

Using XOR :-

```
int n = arr.size();
int ans = 0;

for (int i = 0; i < n; i++){
    ans = ans ^ arr.get(i);
}
return ans;
}
```

TC - O(N)
SC - O(1)

Code:-

```
public static int sigleNonDuplicate(ArrayList<Integer> arr){
    int n = arr.size();
    if (n==1) return arr.get(0);

    for (int i=0; i<n; i++){
        if (i==0){ // check for the first index
            if (!arr.get(i).equals(arr.get(i+1))){
                return arr.get(i);
            }
        }
        else if (i == n-1){
            if (!arr.get(i).equals(arr.get(i-1))){
                return arr.get(i);
            }
        } else {
            if (!arr.get(i).equals(arr.get(i-1)) &&
                !arr.get(i).equals(arr.get(i+1))){
                return arr.get(i);
            }
        }
    }
    return -1;
}
```

TC - O(N)
SC - O(1)

(Optimal):- Binary Search -

arr[] = [1,1,2,2,3,3,4,5,5,6,6]

(1,1) (2,2) (3,3) (5,5) (6,6)
Even odd Even Odd Even Odd Odd Even Odd Even

(Even, Odd) → Element is on right half, x left
(Odd, Even) → Element is on left half, x right

① Write separate code for the first & the last Element.

if (arr[0] != arr[1]) return arr[0];
if (arr[n-1] != arr[n-2]) return arr[n-1]

② [1, 1, 2, 2, 3, 3, 4, 5, 5, 6, 6]
0 1 2 3 4 5 6 7 8 9 10
low mid high

mid → right is same &
I am standing at an even index and my right element is same →
That means I am on the left part
Eliminate left part
· low = mid + 1

③ [1, 1, 2, 2, 3, 3, 4, 5, 5, 6, 6]
0 1 2 3 4 5 6 7 8 9 10

low mid high

mid → right → same mid = $\frac{5+9}{2}$ → (7) ✓

we are on the right part of sigle element (odd, even) index

Eliminate the right part
high = mid - 1

④ [3, 4, 5, 5]
5 6 7 8

low high
mid

mid → left → same
we are on the left part

Eliminate left part
low = mid - 1

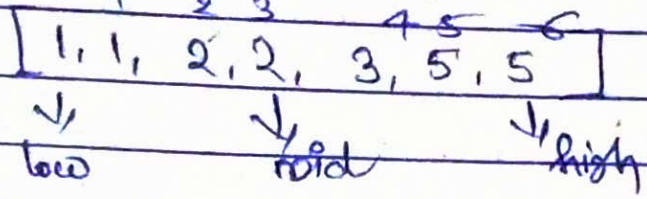
⑤ [4]

high
low
mid

mid = ans :
Sigle element

① Even, Odd (index) concept is present in this question.

Ex.



① mid % 2 == 1 && mid-1 == mid
(even, odd).

eliminate the left part
low = mid+1

② 4 5 6
[3, 5, 5]
low mid high

mid % 2 == 1 && mid+1 == mid
(odd, even)

eliminate right part.
high = mid-1

③ [3]
low
high
mid

mid != mid-1 &&
mid != mid+1

return nums mid) ⇒ 3 Ans.

Code:-

```
public Static int SingleNonDuplicate(int arr[]){
    int n= arr.length;

    // Edge case
    if (n==1) {
        return arr[0];
    }

    if (arr[0] != arr[1]) {
        return arr[0];
    }

    if (arr[n-1] != arr[n-2]) return arr[n-1];

    int low = 1, high = n-2;
    while (low <= high){
        int mid = low + (high-low)/2;

        if (arr[mid] != arr[mid+1] &&
            arr[mid] != arr[mid-1]) {
            return arr[mid];
        }

        if (mid % 2 == 1 && arr[mid] == arr[mid-1]
            || mid % 2 == 0 && arr[mid] == arr[mid+1]){
            low = mid+1;
        } else {
            high = mid-1;
        }
    }
    return -1
}
```

TC- O(logN)
SC- O(1)

(12) Find peak Element - arr[i-1] < arr[i] > arr[i+1]

Ex. 01 arr[] = [1, 2, 3, 4, 5, 6, 7, 8, 5, 1] ans = 8

Ex. 02 arr[] = [1, 2, 1, 3, 5, 6, 4] ans = 2, 5

Ex. 03 arr[] = $-\infty$ [1, 2, 3, 4, 5] $-\infty$ ans = 5

Ex. 04 arr[] = $-\infty$ [5, 4, 2, 3, 1] $-\infty$ ans = 5

Brute force - given $-\infty$ in the first & in the last of the array.

```
for (int i = 0; i<n; i++){
    if ((i==0) || arr[i-1] < arr[i]) &&
       ((i==n-1) || arr[i] > arr[i+1])) {
        return i;
    }
}
```

TC - O(N)
SC - O(1)

Ex: ① [1, 2, 1, 3, 5, 6, 4]
(0 1 2 3 4 5 6)
low ↓ mid ↓ high

nums[mid] < nums[mid+1]
3 < 5

low = mid + 1

② [5, 6, 4]
(4 5 6)
low ↓ mid ↓ high

nums[mid] > nums[mid-1]
&& nums[mid] > nums[mid+1]

return nums[mid]; ⇒ 6 Ans.

Dry run optimal :-

Sr ⇒ [1, 2, 3, 4, 5, 6, 7, 8, 5, 1] Find peak element.

(1) nums[0] = 1 > (X) nums[1] = 2

(2) nums[n-1] = 1 > (X) nums[n-2] = 5

(3) mid = 1+8 ⇒ 9/2 ⇒ 4

nums[4] = 5 (5 > 4) but (5 < 6)

nums[mid] > nums[mid-1]

low = mid+1

(4) [6, 7, 8, 5] 5+8 ⇒ 13/2 ⇒ 6

nums[mid] == 7 (7 > 6) but (7 < 8)

nums[mid] > nums[mid-1]

low = mid+1

(5) [8, 5]

nums[mid] == 8 (8 > 7) and (8 > 5)

peak element is 8

Returning index here ←

PAGE NO.
DATE:

Code:- Binary Search

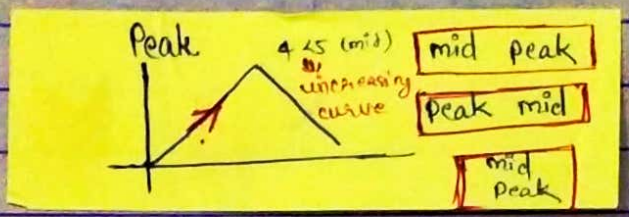
```
if (nums.length == 1) return 0;

int n = nums.length;
// if 0th & n-1th idx is the peak ele
if (nums[0] > nums[1]) return 0;
if (nums[n-1] > nums[n-2]) return n-1;

int low = 1;
int high = n-2;

while (low <= high) {
    int mid = low + (high-low)/2;
    if (nums[mid] > nums[mid-1] &&
        nums[mid] > nums[mid+1]) {
        return mid;
    }
    else if (nums[mid] < nums[mid-1]) {
        high = mid-1;
    }
    else if (nums[mid] < nums[mid+1]) {
        low = mid+1;
    }
    else {
        low = mid+1
    }
}

return -1; // dummy return statement
}
```



(nums[mid] > nums[mid-1]) ⇒ low = mid+1

(nums[mid] < nums[mid+1]) ⇒ high = mid-1

{5 1 2}
0
low point

TC- O(logN)
SC- O(1)

(13) Floor in Sorted Array –

input :- arr() = [1, 2, 8, 10, 11, 12, 19] k = 0

output = -1

input :- arr() = [1, 2, 8, 10, 11, 12, 19] , k = 5

output = 1 (index return)

DRY RUN:- [1, 2, 8, 10, 11, 12, 19] , k = 5

(1) low mid high

mid = 0 + 6 / 2 ⇒ 3

mid = 3 → (10) (5) ~

high = mid - 1

(2) [1, 2, 8, 10] 0 + 3/2 ⇒ 1

low mid high

mid ⇒ 0 + 2 ⇒ 1 (1) ~

2 < (5) ans ⇒ 1 (index no.)

low = mid + 1

(3) [1, 2, 8, 10]

high, mid, low

⇒ 8 > 5, return ans

PAGE NO
DATE:

Code :-

```
int n = arr.length;
int low = 0;
int high = n-1;
int ans = -1;

while ( low <= high ) {

    int mid = ~~low~~ + low + (high - low) / 2;

    if ( arr[mid] <= k ) {
        ans = mid;
        low = mid + 1;
    }
    else {
        high = mid - 1;
    }
}

return ans;
```

{ T.C. - O(logN)

S.C. - O(1) }

Binary Search on answers (min) (max) (Range)...

(14) Find square root of a number in log N.

Ex 01: n = 4 ans = 2

Ex 02 n = 11 ans = 3

Ex 03 n = 1 ans = 1

Brute force:- n = 28

28

i = 1
val = 1x1
val < n
ans = 1

i = 2
val = 2x2
val < n
ans = 2

i = 3
val = 3x3
val < n
ans = 3

i = 4
val = 4x4
val < n
ans = 4

i = 5
val = 5x5
val < 5
ans = 5

i = 6
val = 6 x 6 → 36
~~val~~
36 > n
else
break.

return
ans

⇒ ans = 5

Code:-

```
int ans = 0;
for (long i = 1; i<n; i++){
    long val = i*i;
    if (val <= (long) n){
        ans = (int) i;
    }
    else{
        break;
    }
}
return ans;
}
```

TC = O(N)
SC - O(1)

Optimal Solution 01 using Built in Square root func-

```
public static int floorSqrt(int n){
    int ans = (int) Math.sqrt(n);
    return ans;
}
```

We are basically using the binary search here.

TC - O(logN)

SC - O(1)

Optimal Solution 02 Using Binary Search -

n = 29 low = 1 high = 29 ans = 0

(1) low <= high

mid = low + high/2 ⇒ 30/2 ⇒ 15

mulval = 15 * 15

high = mid - 1

(2) low = 1 high = 14 ans = 0

low <= high

mid = 1 + 14 ⇒ 15/2 ⇒ 7

val = 7 * 7 ⇒ 49

PAGE NO.
DATE: / /

49 > n high = mid - 1

(3) low = 0 high = 6 (ans = 9)
mid = 6/2 ⇒ 3
val = 3 * 3 ⇒ 9
9 < n ans = 9
low ⇒ mid + 1

(4) low = 4 high = 6
mid = 10/2 ⇒ 5
~~ans = 0~~ val = 5 * 5
25 < n
ans = 5
low = mid + 1

(5) low = 6 high = 6
mid ⇒ 12/2 ⇒ 6
val = 6 * 6 ⇒ 36
36 > n
low > high (stop)
return ans;

(2) ans ⇒ 5

Code:

```
int low = 1, high = n;
int ans = 0;
while (low <= high) {
    int mid = low + (high - low) / 2;
    long val = mid * mid;

    if (val <= (long) n) {                    // Eliminate left half
        ans = mid;
        low = (mid + 1);
    }
    else {
        high = (mid - 1);
    }

}

return ans;
}
```

TC - O(logN)
SC - O(1)

(15) Find the Nth root of an integer :-

Ex:01 n = 2, m = 9 output = 3
[b = 2 exp = 9]

Ex:02 n = 3, m = 9 output = -1

Ex:03 n = 1, m = 1.4 output = 14

```
public static long func(int b, int exp){
    long ans = 1;
    long base = b;
    while(exp > 0){
        if(exp % 2 == 1){
            exp--;
            ans = ans * base;
        }
        else{
            exp /= 2;
            base = base * base;
        }
    }
    return ans;
}
public static int NthRoot(int n, int m){
    for(int i = 1; i <= m; i++){
        long val = func(i, n);
        if(val == (long)(m)) return i;
        else if(val > (long)m) break;
    }
    return -1;
}
```

TC - O(M)
SC - O(1)

Brute force Dry-Run:-

⇒ b = 2 (n) exp = 9 (m)

① i = 1 — ~~(m<=)~~ j = 1

func (1, 2)

ans = 1, base = ~~1~~

while (9 > 0) {

exp %. 2 == 0

(9 % 2) == 0

~~exp = 8~~ base = 1

~~ans = 1 * 2~~

~~ans = 2~~

}

~~8 > 0~~

~~exp %. 2 == 0~~

~~exp = 8/2 ⇒ 4~~

~~base = 1 * 1 ⇒ 1~~

1

470

exp %. 2 == 0

exp = 4/2 ⇒ 2

Base = ~~1 * 1 ⇒ 1~~

1 * 1 ⇒ 1

270

exp %. 2 == 0

exp = 2/2 = 1

base = ~~1 * 1 = 1~~

1 * 1 ⇒ 1

② 170

1 %. 2 == 1

exp --; = 0

ans = ans * base

= 2 * ~~2~~ 1

⇒ 2

```
func (int i, int n) {
    int ans = 1;
    for(int j=1; j<=n; j++){
        ans *= i;
    }
    return ans;
}
```

TC - M * N

SC - O(1)

- Optimal Dry-Run :

n = 2 m = 9

① while (low <= high) low = 0, high = 9

mid = 0+9/2 ⇒ 4

int val = func (mid, n, m);

↳ 4*4 ⇒ 16

16 ⩾ m return = 2

high = mid - 1

② mid = 0+3 ⇒ 3/2 ⇒ 1

int val = func (mid, n, m);

↳ 1*1 ⇒ 1

1 < m low = mid+1 return = 0

③ mid = 2+3 ⇒ 5/2 ⇒ 2

int val = func (mid, n, m) ⇒ 2*2 ⇒ 4

4 < m low = mid+1 return = 0

④ mid = 3+3 ⇒ 6/2 ⇒ 3

int val = func (mid, n, m) ⇒ 3*3 ⇒ 9

9 >= m ⇒ return = 1

return (3) ⇒ Ans.

Optimal (Binary Search):-

```
// Return 1 if mid == m
// Return 0 if mid < m
// Return 2 if mid > m

public static int func(int mid, int n, int m){
    long ans = 1;
    for(int i = 1; i <= n; i++){
        ans = ans * mid;
        if(ans > m) return 2;
    }
    if(ans == m) return 1;
    return 0;
}
```

TC- O(logN)
SC- O(1)

```
public static int NthRoot(int n, int m){
    int low = 1; int high = m;

    while(low <= high){
        int mid = (low + high)/2;
        int midN = func(mid, n, m);

        if(midN == 1){
            return mid;
        }else if(midN == 0){
            low = mid+1;
        }else{
            high = mid-1;
        }
    }
    return -1;
}
```

PAGE NO
DATE

(16) Koko Eating Bananas -

Ex01: Input :- piles = [3,6,7,11], h = 8 output = 4 ⇒ k

Ex02: Input :- piles = [30,11,23,4,20], h = 5 output = 30

Brute force :- DRY RUN

① Input = piles = [3, 6, 7, 11] h = 8

k=2 → 2 banana/hrs

2 3 4 6 ⇒ 15 hrs > 8 hrs

✗ (not acceptable)

② Input :- piles = [3, 6, 7, 11] h = 8

k=3 → 3 banana/hrs

1 + 2 + 3 + 4 ⇒ 10 hrs > 8

✗ (not acceptable)

③ Input :- piles = [3, 6, 7, 11] h = 8

k=4 → 4 banana/hrs

1 + 2 + 2 + 3 ⇒ 8

ans = 4

Brute force Code:-

```
public Static int findMax(int arr[]) {          → O(N)
int max = Integer.MIN_VALUE;
int n = v.length;

for(int i=0; i<n; i++) {
    max = Math.max(max, arr[i]);
}
return max;
}
```

TC- O(max(arr) * N)
SC- O(1)

```
public Static int calculateTotalHours(int arr[], int hourly) {
    int totalH = 0;                              → O(N)
    int n = arr.length;

    for(int i=0; i<n; i++) {
        totalH += Math.ceil((double)(arr[i]) / (double)
                  (int)(hourly));
    }

    return totalH;
}

public Static int minRateToEatBananas(int arr[], int h) {
    int max = findMax(arr);  → O(N)

    for(int i=1; i<=max; i++) {  → O(max)
        int reqTime = calculateTotalHours(arr, i);
        if(reqTime <= h) {  → O(N)
            return i;
        }
    }
    return max;
}
```

TC => O(N) + O(max * N)

- Optimal Dry Run:-

Ex:- [3, 6, 7, 11] hrs = 8

① Run loop from 1 to 11 (max)

② low = 1 high = 11

③ while (low <= high)

mid = $\frac{1+11}{2}$ ⇒ 12/2 ⇒ 6

findHrs (piles, 6)

~~3/6 = 0~~ 3/6 = 1 6/6 = 1 7/6 = 2, 11/6 = 2

⇒ 1+1+2+2 ⇒ 6 hrs.

6 <= 8 high = mid-1

④ low = 1 high = 5

mid = 1+5/2 ⇒ 6/2 ⇒ 3

findHrs (piles, 3)

3/3 = 1, 6/3 ⇒ 2, 7/3 = 3, 11/3 = 4

⇒ 1+2+3+4 ⇒ 10

low = mid+1

(5) low = 4 high = 5

mid = 4+5/2 ⇒ 9/2 ⇒ 4

findHrs (piles, 4)

3/4 = 1, 6/4 = 2, 7/4 = 2, 11/4 = 3

⇒ 1+2+2+3 ⇒ 8 hrs

8 <= 8

high = ~~mid+1~~; mid-1;

(6) low = 4, high = ~~5~~ 3

low > high (X) stop.

low = 4 ⇒ Ans.

PAGE NO.
DATE: / /

Code:- Binary Search :-

```
public Static int FindMax (int arr[]){
    int max = Integer.MIN_VALUE;
    int n = arr.length;

    for (int i = 0; i<n; i++){
        max = Math.max (max, arr[i]);
    }

    return max;
}
```

→ O(n)

```
public Static int calculateTotalHours (int arr[], int hourly){
    int totalH = 0;
    int n = arr.length;

    for (int i=0; i<n; i++){
        totalH += (int) (Math.ceil ((double)(arr[i]) /
                                    (double) (hourly)));
    }

    return totalH;
}
```

(n)

```
public Static int minimumRateToEatBananas (int arr[],
                                           int h){
    int low = 1, high = findMax (arr);
```

→ O(n)

```
    while (low <= high){
        int mid = low + (high-low)/2;
```

→ (max/2)

```
        int totalH = calculateTotalHours (arr, mid);
```

→ O(n)

```
if( totalH <= h) {
        high = mid-1; // if in mid hrs koko
    }                 can eat banana so it is possible that
                      she can eat banana in less hrs too
    else {
        low = mid+1; // if koko can not
    }                   eat banana in mid hrs
                    so this speed is too slow for
}                   her, increase speed.
return low;
}
```

O(N) + O(N * log(max))

TC- O(N * log(max(arr)))

SC- O(1)

why max?

Because, we are traversing from 1 to max in log time.

adjacent flowers required

(17) Minimum days to make M bouquets -

Ex. bloomDay = [1, 10, 3, 10, 2], m = 3, k = 1
output = 3
m = no. of bouquets

Ex. bloomDay = [1, 10, 3, 10, 2], m = 3, k = 2
output = -1

Ex. bloomDay = [7, 7, 7, 7, 12, 7, 7], m = 2, k = 3
output = 12

Brute force DRY RUN :-

↑ Bouquets
m = 2 k = 3

- [7, 7, 7, 7, 12, 7, 7]

On 12th day (3 consecutive) k = 3 — Every flowers will be bloomed

Ex. [7, 7, 7, 7, 12, 7, 7] (0 1 2 3 4 5 6), m = 2, k = 3

m * k ⇒ 6 < 7

min = 7 max = 12

i = min → max i = 7

Iteration 01:- possible (bloomDay, m, k, i)

cnt = 0, noOf_B = 0

nums[0] = 7 == 7 cnt++ cnt = 1
1 = 7 <= 7 cnt = 2
2 = 7 <= 7 cnt = 3
cnt == 3 . noOfB = 1

noOfB < m (2)

Iteration 02:- $i = 8$ possible (bloomDay, m, k, i)

possible ⇒ noOfB = 0, cnt = 0

$i = 0$	(7) $\not<$ 8	cnt = 1
$i = 1$	(7) $\not<$ 8	cnt = 2
$i = 2$	(7) $\not<$ 8	cnt = 3 == k, noOfB = 1, cnt =
$i = 3$	(7) $\not<$ 8	cnt = 1
$i = 4$	(12) $\not<$ 8	cnt = 0
$i = 5$	(7) $\not<$ 8	cnt = 1
$i = 6$	(7) $\not<$ 8	cnt = 2

(1) noOfB < m) (2)

Iteration 03:- $i = 9$ possible (bloomDay, m, k, i)

possible ⇒ noOfB = 0, cnt = 0

$i = 0$	(7) $\not<$ 9	cnt = 1.
$i = 1$	(7) $\not<$ 9	cnt = 2
$i = 2$	(7) $\not<$ 9	cnt = 3 == k noOfB = 1 cnt = 0
$i = 3$	(7) $\not<$ 9	cnt = 1
$i = 4$	(12) $\not<$ 9	cnt = 0
$i = 5$	(7) $\not<$ 9	cnt = 1
$i = 6$	(7) $\not<$ 9	cnt = 2

(1) noOfB < m (2)

Brute force Code:-

```
public static boolean possible(int arr[], int day, int m,
                               int k) {
    int n = arr.length;
    int cnt = 0;
    int noOfB = 0;

    for (int i = 0; i<n; i++) {  → O(N)
        if (arr[i] <= day) {
            cnt++;
            if (cnt == k) noOfB++; cnt = 0;
        } else {
            noOfB += (cnt / k);
            cnt = 0;
        }
    }
    noOfB += (cnt / k);
    return noOfB >= m;
}
```

```
public static int roseGarden(int arr[], int k,
                             int m) {
    long val = (long) m * k;
    int n = arr.length;
    if (val > n) return -1;

    int min = Integer.MAX_VALUE;
    int max = Integer.MIN_VALUE;
```

```
for(int i=0; i<n; i++) {                    → O(N)
    min = Math.min(min, arr[i]);
    max = Math.max(max, arr[i]);
}

for(int i=min; i<=max; i++) {               → (min-max)
    if(possible(arr, i, m, k)) {
        return i;                           → O(N)
    }
}
return -1;
}
```

~~O(max-min) * O(N) + O(N)~~ ≈ -

TC - O(max(arr) - min(arr+1) * N
SC - O(1)

PAGE NO
DATE

Optimal (BS - DRY RUN Here):-

~~05~~ ~~Combination Sum - II~~

Ex. [7, 7, 7, 7, 12, 7, 7] m = 2 k = 3
0 1 2 3 4 5 6

Iteration 01 :- min = 7 max = 12 i = 7

possible (bloomDay, m, k, i)

cnt = 0, noOfB = 0

i = 0 = 7 <= 7 cnt = 1
i = 1 = 7 <= 7 cnt = 2
i = 2 = 7 <= 7 cnt = 3 ⇒ if (cnt == k) noOfB = 1, cnt = 0
i = 3 = 7 <= 7 cnt = 1
i = 4 = 12 > 7 cnt = 0
i = 5 = 7 <= 7 cnt = 1
i = 6 = 7 <= 7 cnt = 2

noOfB < m (2)

Iteration 02 :- i = 8 possible (bloomDay, m, k, i)

cnt = 0, noOfB = 0

i = 0 = 7 <= 8 cnt = 1
i = 1 = 7 <= 8 cnt = 2
i = 2 = 7 <= 8 cnt == 3 == k noOfB = 1 ⇒ cnt = 0
i = 3 = 7 <= 8 cnt = 1
i = 4 = 12 > 8 cnt = 0
i = 5 = 7 <= 8 cnt = 1
i = 6 = 7 <= 8 cnt = 2

(1) noOfB < m (2)

Iteration 03 :- i = 8, possible()
cnt = 0, noOfB = 0

i = 0 = 7 <= 8
i = 1 = 7 <= 8
i = 2 = 7 <= 8 cnt = 3 noOfB = 1 cnt = 0
i = 3 = 7 <= 8 cnt = 1
i = 4 = 12 > 8 cnt = 0
i = 5 = 7 <= 8 cnt = 1
i = 6 = 7 <= 8 cnt = 2

(1) noOfB < m (2)

Iteration 04 :- i = 10, possible()
cnt = 0, noOfB = 0

i = 0 = 7 <= 10 = cnt = 1
i = 1 = 7 <= 10 = cnt = 2
i = 2 = 7 <= 10 = cnt = 3 = k
i = 3 = 7 <= 10 = cnt = 1
i = 4 = 12 > 10 = cnt = 0
i = 5 = 7 <= 10 = cnt = 1
i = 6 = 7 <= 10 = cnt = 2

noOfB(1) < m(2)

Iteration 05 :- i = 11, possible()
cnt = 0, noOfB = 0

i = 0 = 7 <= 11 cnt = 1
i = 1 = 7 <= 11 cnt = 2
i = 2 = 7 <= 11 cnt = 3 = k
i = 3 = 7 <= 11 cnt = 1
i = 4 = 12 > 11 cnt = 0
i = 5 = 7 <= 11 cnt = 1
i = 6 = 7 <= 11 cnt = 2

(1) noOfB < m(2)

Iteration 06 :- i = 12, possible()
cnt = 0, noOfB = 0

i = 0 = 7 <= 12 cnt = 1
i = 1 = 7 <= 12 cnt = 2
i = 2 = 7 <= 12 cnt = 3 = k (1)
i = 3 = 7 <= 12 cnt = 1
i = 4 = 12 <= 12 cnt = 2
i = 5 = 7 <= 12 cnt = 3 (2)
i = 6 = 7 <= 12 cnt = 1

noOfB(2) >= m

return i = = 12

Ans

PAGE NO.
DATE

Optimal Code:-

```
public static boolean possible(int arr[], int day,
        int m, int k) {
    int n = arr.length;
    int cnt = 0;
    int noOfB = 0;

    for(int i = 0; i<n; i++) {
        if(arr[i] <= day) {
            cnt++;
            if(cnt == k) noOfB++;
                cnt = 0;
        } else {
            cnt = 0;
        }
    }
    return noOfB >= m;
}
```

```
public static int roseGarden(int arr[],
        int k, int m) {
    long val = (long) m * k;  → might exceed
    int n = arr.length;

    if (val > n) {
        return -1;
    }
```

```
int min = Integer.MAX_VALUE;
int max = Integer.MIN_VALUE;

for (int i = 0; i<n; i++) {                    → O(N)
    min = Math.min(min, arr[i]);
    max = Math.max(max, arr[i]);
}

int low = min;
int high = max;

while (low <= high) {                          → O(log(max-min)+1)
    int mid = low + (high - low) / 2;

    if (possible(arr, mid, n, k)) {
        high = mid - 1;                        → O(N)
    }
    else {
        low = mid + 1;
    }
}
return low;
}
```

O(N) + O(log(max-min)+1) * N)

TC - O(log(max(arr) - min(arr) + 1) * N

SC - O(1)

~~14.~~ ~~Permutations~~ —

18. Find the Smallest Divisor —

Ex. arr[] = [1, 2, 5, 9] threshold = 6

ans = 5

① $\frac{1}{4}$ $\frac{2}{4}$ $\frac{5}{4}$ $\frac{9}{4}$ → Let it is not possible divisor

↓ ↓ ↓ ↓ should be lesser...

1 + 1 + 2 + 3 ⇒ 7 <= 6 (✗)

② $\frac{1}{5}$ $\frac{2}{5}$ $\frac{5}{5}$ $\frac{9}{5}$ → It is lesser than the threshold

↓ ↓ ↓ ↓

1 + 1 + 1 + 2 ⇒ 5 <= 6

ans ⇒

Brute Force DRY RUN :—

Ex. [1, 2, 5, 9] threshold = 6
0 1 2 3

Iteration 01 ⇒ i = 1 isThreshold (nums, i, threshold);

Sum = 0,

i = 0 ⇒ 1/1 = 1 Sum = 1

i = 1 ⇒ 2/1 = 2 Sum = 3

i = 2 ⇒ 5/1 = 5 Sum = 8

i = 3 ⇒ 9/1 = 9 Sum = 17

(17) Sum >= 6

Iteration 02:- $i = 2$ isThreshold (nums, i, threshold)

Sum = 0,

$i=0$	$1/2$	$= 1$	Sum = 1
$i=1$	$2/2$	$= 1$	Sum = 2
$i=2$	$5/2$	$= 3$	Sum = 5
$i=3$	$9/2$	$= 5$	Sum = 10

Sum (10) > threshold (6)

Iteration 03:- $i = 3$ isThreshold (nums, i, threshold)

Sum = 0

$i=0$	$1/3$	$= 1$	Sum = 1
$i=1$	$2/3$	$= 1$	Sum = 2
$i=2$	$5/3$	$= 2$	Sum = 4
$i=3$	$9/3$	$= 3$	Sum = 7

(7) Sum > 6

Iteration 04 $i = 4$ isThreshold (nums, i, threshold)

Sum = 0

$i=0$	$1/4$	$= 1$	sum = 1
$i=1$	$2/4$	$= 1$	sum = 2
$i=2$	$5/4$	$= 2$	Sum = 4
$i=3$	$9/4$	$= 3$	Sum = 7

(7) Sum > 6

return ans

Iteration 05 $i = 5$ isThreshold (nums, i, threshold).

Sum = 0

$i=0$	$1/5$	$= 1$	sum = 5
$i=1$	$2/5$	$= 1$	
$i=2$	$5/5$	$= 1$	
$i=3$	$9/5$	$= 2$	

5 < 6

Brute force :—

```
public static int smallestDivisor(int arr[], int limit){
    int n = arr.length;
    int max = Integer.MIN_VALUE;

    for(int i=0; i<n; i++){                    → O(N)
        max = Math.max(max, arr[i]);
    }

    for(int d=1; d<=max; d++){                 → max
        int sum = 0;
        for(int i=0; i<n; i++){                → (N)
            sum += Math.ceil((double)(arr[i])
                    /(double)(d));
        }
        if(sum <= limit){
            return d;
        }
    }
    return -1;
}
```

O(max * N) + O(1)

TC - O(max(arr) * N)

SC - O(1)

DRY RUN (BS) :-

Ex. [1, 5, 2, 7, 8, 9, 4] threshold = 5
0 1 2 3 4 5 6
low = 1 high = 9

Iteration 01 :- mid = 10/2 ⇒ 5 isThreshold()

i = 0 = 1/5 = 1 = sum = 1
i = 1 = 5/5 = 1 = sum = 2
i = 2 = 2/5 = 1 = sum = 3
i = 3 = 7/5 = 2 = sum = 5
i = 4 = 8/5 = 2 = sum = 7
i = 5 = 9/5 = 2 = sum = 9
i = 6 = 4/5 = 1 = sum = 10

(10) sum > 5 ✱ low = mid+1

Iteration 02 ⇒ low = 6, high = 9
mid = 6+9/2 ⇒ 15/2 ⇒ 8

i = 0 = 1/8 = 1 = sum = 1
i = 1 = 5/8 = 1 = sum = 2
i = 2 = 2/8 = 1 = sum = 3
i = 3 = 7/8 = 1 = sum = 4
i = 4 = 8/8 = 1 = sum = 5
i = 5 = 9/8 = 2 = sum = 7
i = 6 = 4/8 = 1 = sum = 8

(8) sum > 5 low = mid+1

Iteration 03 ⇒ low = 9, high = 9
mid = 9+9/2 ⇒ 9

1/9 = 1, 5/9 = 1 2/9 = 1 7/9 = 1; 8/9 = 1

4/9 = 1 9/9 = 1 sum ⇒ 8 > 7 return -1

PAGE NO.
DATE

BS (Code) :-

~~66.~~ ~~Permutations — II~~

```
public static int sumByD(int arr[], int div){
    int n = arr.length;

    int sum = 0;
    for(int i = 0; i<n; i++){
        sum += Math.ceil ((double)(arr[i])
                / (double)(div));
    }

    return sum;
}
```

```
public static int smallestDivisor(int arr[], int limit){
    int n = arr.length;
    if (n > limit) return -1;

    int max = Integer.MIN_VALUE;
    for(int i = 0; i<n; i++){
        max = Math.max(max, arr[i]);
    }
    int low = 1;                          O(N)
    int high = max;

    while(low <= high){ → log(max)

        int mid = low + (high - low) / 2;
```

```
if (sumByD(arr, mid) <= limit) {
    high = mid-1;                    (N)
}
else {
    low = mid+1;
}
}
return low;
}
```

if sum is greater then I need to increase my no. count for divisor

⇒ O(log(max) * N) + O(N)

TC- O(log(max(arr)) * N)

SC - O(1)

PAGE NO.
DATE: / /

Least

19. Capacity to Ship Packages within D days -

weights[] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] days = 5
0 1 2 3 4 5 6 7 8 9

Dry Run:- Iteration 01:- max → sum (range)
DRY RUN:-

i = 10 DaysReq(weights, i);
load = 0, days = 1

i = 0 ⇒ 0 + 1 ⇒ 1 ⇒ load days = 1

i = 1 ⇒ 1 + 2 ⇒ 3 ⇒ load
i = 2 ⇒ 3 + 3 ⇒ 6 ⇒ load
i = 3 ⇒ 6 + 4 ⇒ 10 ⇒ load days = 1
i = 4 ⇒ 10 + 5 ⇒ 15 ⇒ load = 5 days = 2

i = 5 ⇒ 5 + 6 = 11 ⇒ Exceeds.
load = 6 days ⇒ 3

i = 6 ⇒ 6 + 7 ⇒ 13 ⇒ Exceeds
load = 7 days ⇒ 4

i = 7 ⇒ 7 + 8 ⇒ 15 ⇒ Exceeds
load = 8 days = 5

i = 8 ⇒ 8 + 9 ⇒ 17 ⇒ Exceeds days = 6
load = 9.

i = 9 ⇒ 9 + 10 ⇒ 19 Exceeds days = 7

7 > 5

Iteration - 02